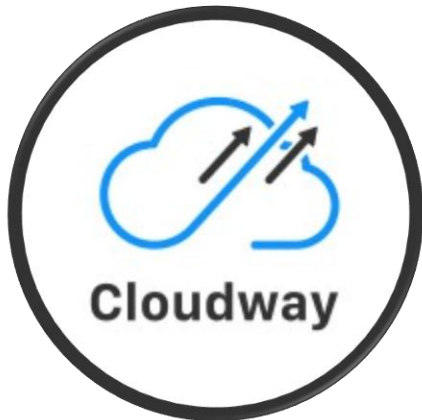




CLOUDWAY

TEAM: ICARUS-M2

Project Documentation

Sybrein Backx,
Evert Schol,
Maximus Mukiza,
Paschal Chukwubuikem Ifewulu,
Ravin Shalmashi,
Thomas Van Sande,
Youssef Nait Salah



Table of content

Table of content	2
Introduction	4
1. AI	5
1.1 Overview	5
1.2 Data Architecture	5
1.3 Implementation Details	6
1.4 Evaluating model response	8
2. APP	9
2.1 Overview	9
2.2 Database Schema	9
2.3 Events and Worker Architecture	13
2.4 Events	14
2.1 MessageReceived	14
2.2 FetchFacts	14
2.3 CallFirstModel	14
2.4 CallLogicUnit	14
2.5 ReadabilityPostProcess	15
2.6 FinalizeRequest	15
3. API Table	15
3. CCS	15
3.1 Icarus infrastructure: AWS deployment	15
3.2 Architecture	16
3.2.1 How it's built	16
3.2.2 Network layer	16
3.2.3 Application layer	16
3.2.4 Data layer	17
3.3 Component relationships and data flow	17
3.3.1 Key design decisions:	17
3.3.2 Architectural decisions	17
3.4 Backend application	19
3.4.1 What's inside	19
3.4.2 Cool things we've done	19
3.5 Frontend application	19
3.5.1 How it stays flexible	20
3.6 State management strategy	20
3.6.1 Why we like the HTTP backend	20
3.6.2 Keeping things locked and safe	21
3.6.3 Our workspace strategy	21
3.6.4 Why we use multiple environments	21

3.7 Security by design	22
3.7.1 Encrypting your data	22
3.7.2 Permissions that make sense	22
3.7.3 Scanners that watch your back	22
3.8 Network security	23
3.9 Operational guide	23
3.9.1 Local setup	23
3.9.2 What you'll need	23
3.9.3 Your environment variables	23
3.9.4 How to initialize	23
3.9.5 Your daily workflow	24
3.10 CI/CD pipeline	24
3.10.1 Gate 1: Verification (automatic)	24
3.10.2 Gate 2: Production promotion (post-merge)	25
3.11 Safety guardrails	25
Cost optimization	25
Cost breakdown	26
Budget controls	26
NAT Gateway monitoring	26
Cost optimization strategies	26
Monitoring and observability	27
Unified monitoring dashboard	27
Cost monitoring and analysis	27
When we'll alert you	27
3.12 Logs and debugging	28
ECS logs	28
Lambda logs	28
VPC flow logs	28
Terraform state	28
3.13 Troubleshooting	29
Common issues you might run into	29
"State is locked"	29
High NAT costs	29
ECS won't deploy	29
CloudFront errors	29
3.14 Glossary	30
Conclusion	30

Introduction

This document provides the technical documentation for the Cloudway project. It describes the architecture, implementation, and integration of the 3 different teams AI, APP and CCS.

The first part of this documentation focuses on the Artificial Intelligence (AI) component. The AI layer is responsible for processing user requests using language models and logic units. It handles prompt processing, validation of outputs, and ensures that generated responses meet predefined quality and reliability standards. The AI system is designed to operate within a controlled environment where results can be monitored, evaluated, and improved.

The second part covers the Application (App) component, which forms the base of the platform. This layer manages backend services, API endpoints, request handling, and system orchestration. It connects the user interface with the AI services and ensures that requests are processed efficiently. The application layer is responsible for logging, monitoring, and maintaining communication between all system components.

The third part describes the CCS component. Security is integrated throughout the platform to guarantee data protection, controlled access, and system integrity. The cybersecurity layer implements authentication, authorization, secure communication, and monitoring mechanisms. It ensures that users can safely interact with the system while protecting sensitive data and preventing unauthorized access.

The Cloudway project was developed through a collaboration of three student teams, each responsible for one of these core domains. By combining AI capabilities, backend engineering, and cybersecurity principles, the project delivers a great and secure integration platform.

1.AI

1.1 Overview

The AI component in our project serves as a bridge between the user input (text, images, documents) and the deterministic logic from the Logic Unit. There are 2 primary AI services. The first is extracting the initial user input, which can be a combination of partial free text, images and documents (Word, PDF, Excel). The AI model tries to extract specific facts from this input that are relevant to the selected policy for the Logic Unit. If the user selected a text model for the integration then there will only be text pre-processing. If the user selected a multi-modal model for the integration then a service will be called that pre-processes both the text and the attachments. When is done pre-processing it returns a JSON structured output and the event-based worker will call the Logic Unit for processing this output.

When the Logic Unit processes this output, it generates a combination of JSON and natural language containing information about whether the claim is approved, denied, or needs more information. It also contains a lot of information about the next steps that should be taken. A second AI service is responsible for making this output more readable for the final user. This is the post-processing part, it takes the raw Logic Unit results and formats them into a professional and readable summary. This output can be returned as text, JSON, PDF, Word, or Excel files. When creating an integration, the user selects the desired output type. If JSON is selected, the user can also modify the JSON schema to define which fields the final output should contain and how the JSON is structured.

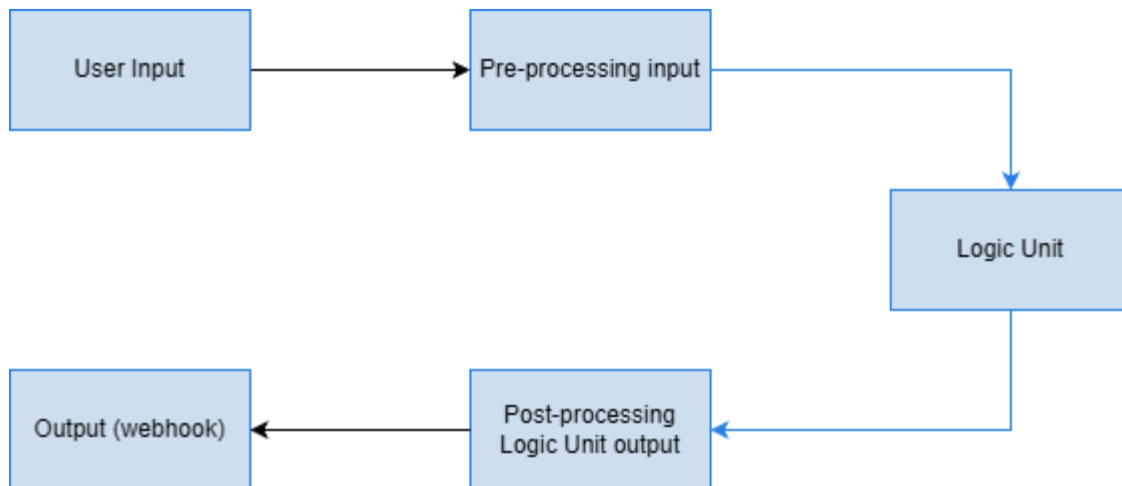
All these events are orchestrated through an event-based worker system. This asynchronous workflow ensures that the application is resilient to API timeouts from Bedrock or the Logic Unit.

For evaluating the quality of the final generated output, we use a AI judge model. This is a model that will compare the Logic Unit output and the final generated output and check for hallucinations, generalizations, lack of information etc.. If the output is in JSON we also validate if all JSON fields are included in the final output.

The AI model used for this project is Amazon Nova Lite V1. This is a foundational model on the AWS platform that we can call using the Amazon Bedrock API. The AI part is done in the backend. This means it's also in the TypeScript language. We are using them as services that can be called so it's easier to have them in the backend instead of making them with Python code and then having to set up an API server so the application can reach the models.

1.2 Data Architecture

Now that we gave an overview of the AI functionalities, we continue with giving a general overview of the data pipeline:



1. **Ingestion user input:** User selects an integration that contains information about the applicable policy, system prompt, pre-processing model and output format. This is done through making a request with an API key that links to the integration. In the text field the user describes the claim and additionally he can also provide images/documents for further proof. The worker also fetches the facts so the AI model knows what it needs to look for.
2. **Pre-processing input:** The AI model analyses the text/images/documents and maps them to the required facts. This will be done in a JSON structure.
3. **Logic Unit receives request:** The Logic Unit is called and the JSON structure of the pre-processing step is included. The Logic Unit will generate a raw natural language decision making output.
4. **Post-processing Logic Unit output:** The raw output of the Logic Unit will be used for post-processing. It transforms this into the requested format so the output is more readable.
5. **The output:** The worker marks the request as completed and if a webhook was configured in the integration it will return the output to this location.

1.3 Implementation Details

With the scope of the AI part defined and the outline of the data pipeline, we can now go over the whole implementation in detail of all the AI functionalities we realized for this project. As mentioned in the overview all AI coding is done in the TypeScript language so it can be easily called in the backend. Since we aren't training a model or fine-tuning it, there is no need to do this in Python and having to create a separate

API server using something like Fastapi. Since Amazon Nova Lite V1 has multi-modal capabilities, we can use this model for all phases of the AI part including the processing of images, text and documents. The specific name of the model used in the code is 'amazon.nova-lite-v1:0'.

The pre-processing stage is handled by the `handleFirstModelPreProcessing` worker function. Here the prompt for the AI model is generated. It combines the custom instructions retrieved from the database of the specific integration (identified by passing along an API key), the previous messages in the same session so context is maintained and the facts retrieved from the Logic Unit. If the selected pre-processing model was a text model, the function 'callTextModel' in the 'CallBedrockModelService.ts' is called. This receives the text input and prompt and processes this. It then is parsed into a JSON structured output along with the metrics of input tokens, output tokens and total tokens for the request.

If the selected pre-processing model was multi-modal, the function 'callMultimodalModel' in the 'CallBedrockModelService.ts' is called. This function has the same process as the text model, but it can also receive documents and images in the input. The images go through some pre-processing. For this we use the sharp library. The images are rescaled so they will always be clear but also puts a cap on the maximum pixels (8000) so that the model input limits are never exceeded. It also makes sure that aspect ratio is always maintained and that the image is always clear. The images are included in the payload as base64 strings. Documents are sanitized by removing the file name extensions. It also uses regex to strip any character that aren't letters, numbers, hyphens or brackets. Spaces are replaced with hyphens to ensure the API request does not fail. After that the raw bytes are attached to the payload. If the file extension is xlsx then we use the exceljs library for processing this. It iterates through every row extracting the values and sanitizes the values by converting them to strings and escaping double quotes. Each row is joined by commas and appended with a newline character. Basically the excel file is transformed into a CSV file. After the first model has been called and it completed the processing, the worker will makes a request to the Logic Unit containing the model response in the payload.

The post-processing stage is responsible for transforming the raw output of the Logic Unit into the final user-facing response, while strictly preserving the original decision. The Logic Unit remains the single source of truth in the system; post-processing does not alter, reinterpret, or override any logical outcome. Instead, once the Logic Unit completes its evaluation, an event-based worker triggered through a webhook callback initiates the readability step. This stage focuses purely on formatting and structuring the result according to the configuration defined for the specific integration.

Each integration defines its own output type, which can be text, JSON, PDF, Word, or Excel. If the selected output type is JSON, the integration may also define a custom JSON schema. This allows the integration owner to control exactly which fields are returned and how the final JSON is structured. If no custom schema is

provided, a default schema is applied. For non-JSON outputs, such as PDF, Word, or Excel, this same default schema is always used internally as an intermediate representation. This ensures that the system maintains a consistent internal structure before transforming the data into the requested format.

The readability model receives the Logic Unit's output together with strict formatting instructions and the applicable schema. Based on this configuration, it generates either plain text or structured JSON that conforms precisely to the defined schema. In the case of file-based outputs (PDF, Word, Excel), the structured JSON is then converted into a professionally formatted document. This guarantees that even document-based outputs are generated from a validated and consistent data structure.

In the current version of the application, since final results are demonstrated directly through webhook responses in the frontend, the PDF, Word, and Excel output options have been removed from the frontend interface. If these formats are manually triggered from the frontend, an error will appear. However, this limitation only applies to the frontend layer. The complete file-generation process is fully implemented and operational in the backend. The system is capable of generating properly formatted PDF, Word, and Excel files, and these can be used in real system-to-system integrations. Example output files will be provided to demonstrate how these final documents are structured and formatted.

[readability-report \(1\).docx](#)
[readability-report \(1\).xlsx](#)

This architecture ensures that the Logic Unit remains authoritative and that output formatting is entirely controlled through integration settings. By separating logical decision-making from presentation and formatting, the system achieves flexibility, consistency, and reliability across multiple output types while maintaining strict control over structure and validation.

1.4 Evaluating model response

We want to make sure that the readable output we generate in the end is correct. No hallucinations should be present, it should contain all the information the logic output generated and not be too general. For this we used two things. In the 'EvaluateReadabilityService.ts' file the 'evaluateReadability' does 2 checks. First it checks, using the zod library, whether the output JSON contains all the required fields. It checks the default JSON schema unless a custom JSON schema has been specified. The second check is an AI judge evaluation. It calls an LLM that compares the raw Logic Unit output and the processed readable output. It gives a score based on faithfulness, clarity, tone and completeness. It specifically checks for

hallucinations or contradictions. If the AI altered the logical decision the score is capped so it remains low. There is also a field where it explains the feedback so you can see what it detected so you don't only rely on a score that the AI itself generates.

To check how well the judge AI performs, we made a unit test called 'EvaluateReadabilityService.test.ts'. In this test we look for edge case scenarios to see if the judge model catches certain scenarios. There is a scenario where there is a lie, a creative one where there is a hallucination and a vague one where we check if the judge detects the output being too vague. It detects all 3 edge cases.

2. APP

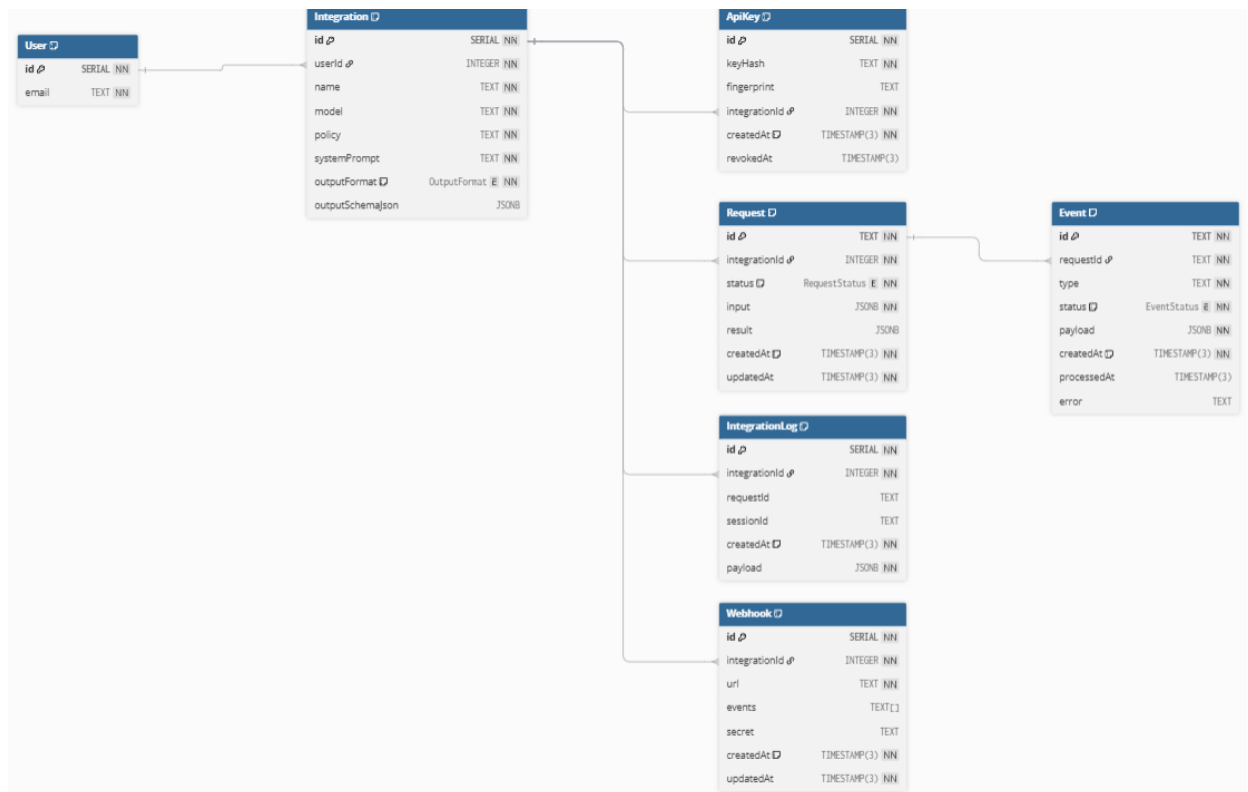
2.1 Overview

The application component of the project is responsible for exposing the system's functionality to end users and external clients in a secure, scalable, and configurable way. While the AI and Logic Unit handle decision-making and content generation, the application layer acts as the orchestration and control layer that manages integrations, requests, persistence, and asynchronous processing.

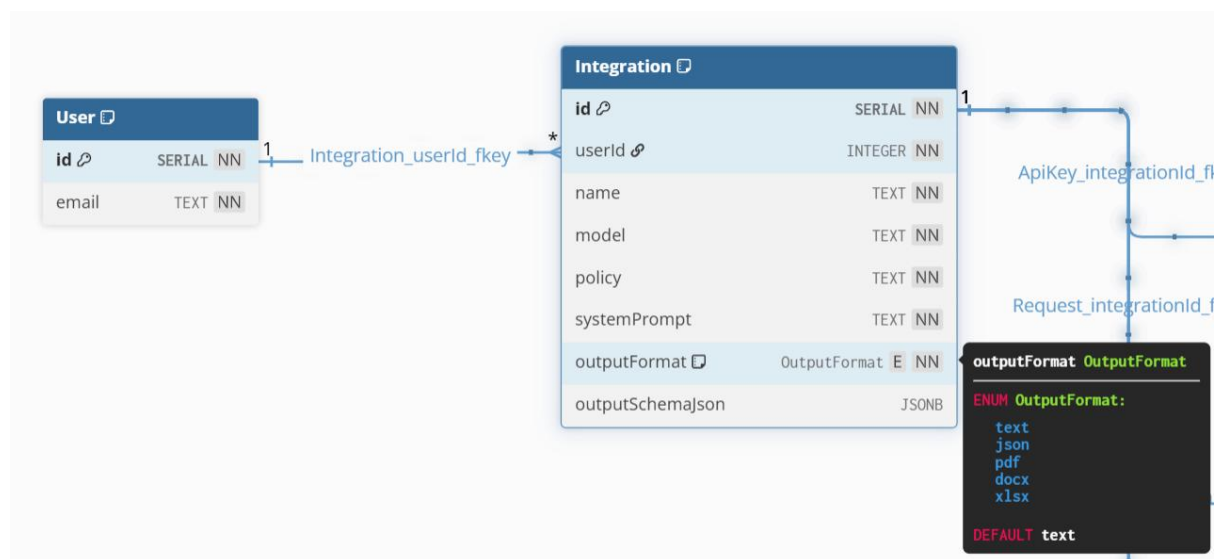
The backend application is implemented in TypeScript using Node.js and Express. It exposes a REST API that allows users to configure integrations, submit requests, and retrieve results. The application also manages authentication, database access, validation, and communication with background workers.

A key design goal of the application layer is separation of concerns. Business logic is isolated into services, validation is handled through schemas, controllers remain thin, and long-running or failure-prone operations are executed asynchronously through workers. This ensures maintainability, testability, and resilience.

2.2 Database Schema



1 Full database schema

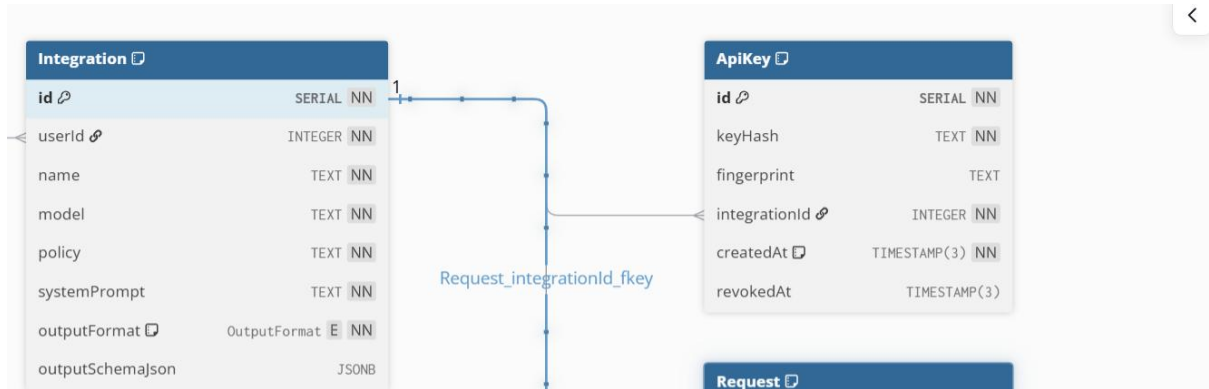


2 User and Integration table

There is one admin user in the system. He can create multiple integrations. An integration holds the configuration options to be used when a request is made. The different fields are:

- Model: Text or multi-modal. It determines the ai model to be used.
- Policy: which policy to use inside the Logic Unit. Which policy facts to load in.
- Systemprompt: configurable string to provide context with the first ai model.
- Outputformat: Selectable format to receive the final response in.

- Outputschemajson: configurable schema when json OutputFormat is selected.

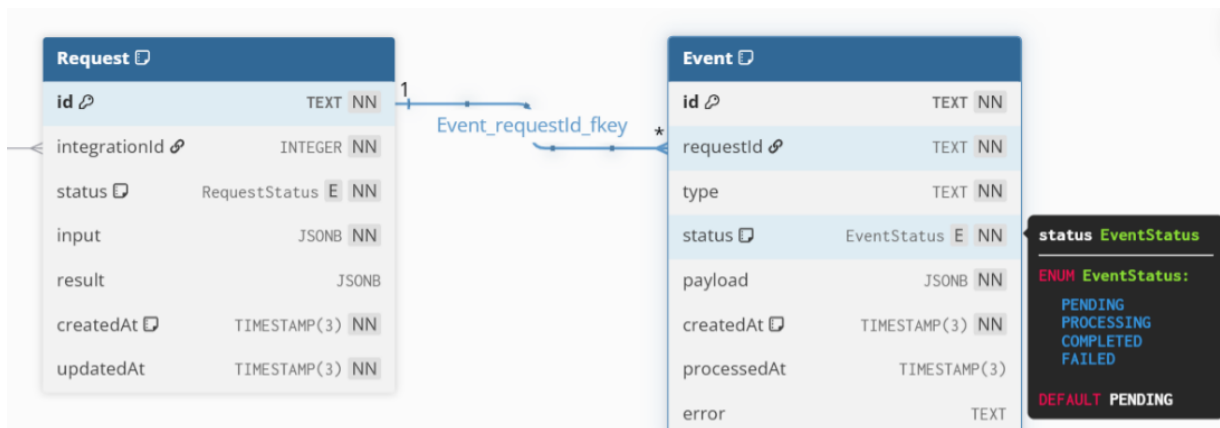


3 Integration and ApiKey table

An integration can contain multiple api keys. Only one however can be active at a time. Keys can be revoked and rotated in the system. The keys are hashed and only the hashed key is being stored for security.

The different fields are:

- Keyhash: hashed api key string
- Fingerprint: identifier of the hashed api key.
- CreatedAt: date of creation. keys are created in 2 ways. At first integration creation and when rotating the key
- RevokedAt: Selectable format to receive the final response in.



4 Request and Event Table

Every Request that is made to the system is cast to the request table. A request is always coupled with an integration. The table holds the input and result of the message that comes in.

The different fields are:

- Status: holds the object RequestStatus with an enumerator over the options Pending, Processing, Completed and Failed.
- Input: The original message that came in from the user calling the system.

- Result: The final output response of fully processed message.

The event table serves as a queue where workloads are stored for the worker. The worker looks at the table in an interval and checks if there are any events with a status of “pending”. The payload of each event is saved and also an error message in case something goes wrong.

The different fields are:

- Type: The name of the event. This is used to know which handler call.
- Status: holds the object EventStatus with an enumerator over the options Pending, Processing, Completed and Failed.
- Payload: Holds all relevant information and the result of the processing step performed on it.
- Error: Holds the error message in case something fails.

IntegrationLog		
id	SERIAL	NN
integrationId	INTEGER	NN
requestId	TEXT	
sessionId	TEXT	
createdAt	TIMESTAMP(3)	NN
payload	JSONB	NN

5 IntegrationLog table

The logging table which stores the payloads from the events of each request.

The different fields are:

- SessionId: The id of the session that matches the id of each request.
- Payload: The result of a processing step in the event cycle

Webhook	
id	SERIAL NN
integrationId	INTEGER NN
url	TEXT NN
events	TEXT[]
createdAt	TIMESTAMP(3) NN
updatedAt	TIMESTAMP(3) NN

6 Webhook Table

An integration can be set up with one or multiple webhooks. At the end of the event cycle exists a loop that posts the end result payload to each webhook that is configured. Messages are only send if the webhook is subscribed to the right trigger. At the moment the two triggers are “Completed” and “failed”. This way it is possible to receive the final result on one url and errors on another.

The different fields are:

- Url: the webhook url that is posted to at the end of the event cycle.
- Events: Set to either "request.completed" or "request.failed". This is used to check a condition if the integration is subscribed to the right trigger event for it to post the message.

2.3 Events and Worker Architecture

Event	Handler
MessageReceived	handleMessageReceived
FetchFacts	handleFetchFacts
CallFirstModel	handleFirstModelPreProcessing
CallLogicUnit	handleCalllogicUnit
ReadabilityPostProcess	handleReadabilityPostProcess
FinalizeRequest	handleFinalizeRequest

The system uses an event-driven worker architecture backed by a persistent job queue.

Each processing step is represented as an Event stored in the database and handled asynchronously by a background worker.

The system processes work using a background worker that operates over a persistent event table. The worker runs at a fixed interval and queries the database for events whose status is set to **PENDING**. When such an event is found, the

worker claims it and updates its status to **PROCESSING** to prevent duplicate handling. The corresponding event handler is then executed.

If the handler completes successfully, the event is marked as **COMPLETED** and processing continues to the next event in the request lifecycle. If an error occurs during execution, the worker automatically retries the event. Each event may be retried up to three times. If all retry attempts fail, the event is marked as **FAILED**, this allows the system to track all processing in the logs and audit the requests.

All external calls, including AI model calls and logic unit executions, are wrapped in a retry mechanism.

2.4 Events

2.1 MessageReceived

The MessageReceived event marks the beginning of the request lifecycle. It is triggered whenever a new request is submitted to the system. An initial event is persisted to the database to ensure the request is recorded. The incoming API key is resolved to get the correct integration configuration, and the request context is initialized so that following processing steps have access to all required metadata.

2.2 FetchFacts

Once the request context has been established, the FetchFacts event is executed. This step resolves the policy associated with the selected integration and loads any specific policy facts required for later processing. A bearer token is generated if it doesn't exist yet or is expired. Then a call is made to the Logic unit to get the facts for a specific policy. These facts are prepared and structured so they can be incorporated into the system prompt used during model execution.

2.3 CallFirstModel

The CallFirstModel event is responsible for invoking the primary AI model. During this step, the system prompt is constructed using the previously loaded facts and the integration's configuration. The configured model is then called, and its response is constrained to a structured JSON format. This structured output is used later in the Logic Unit.

2.4 CallLogicUnit

After the first model execution, the CallLogicUnit event processes the structured JSON output. Again a bearer token is used with the call and the first model response is given to the Logic unit. Then the logic unit applies deterministic transformations, validations, and business. This step produces a normalized intermediate result that is suitable for final presentation and formatting.

2.5 ReadabilityPostProcess

The ReadabilityPostProcess event cleans the intermediate result into its final presentable form. The output produced by the logic unit is sent to a secondary model, which adjusts the content based on the integration's configuration and the desired output format. This step ensures that the final response conforms to the expected presentation requirements, whether the output is plain text, structured JSON, or a document-based format.

2.6 FinalizeRequest

The request lifecycle concludes with the FinalizeRequest event. At this stage, the request is marked as either COMPLETED or FAILED depending on the outcome of the preceding steps. The system then iterates over all webhooks configured for the integration and delivers the final response payload to each registered webhook URL. This event represents the terminal point of processing.

3. API Table

- See API Table pdf file

3. CCS

3.1 Icarus infrastructure: AWS deployment

Welcome to the Icarus infrastructure! You'll use this project to set up and manage a cost-optimized AWS environment using Infrastructure as Code (IaC). We've built this with OpenTofu (a friendly fork of Terraform) to orchestrate everything from your frontend assets in Amazon S3 to your backend services running on Amazon ECS Fargate.

We've unified everything under a single Amazon CloudFront distribution. This doesn't just make things faster, it also eliminates those annoying CORS headaches by serving your frontend and backend from the same domain.

The setup supports a smooth flow from development to production. It includes automated security checks, detailed monitoring, and smart cost-saving features to keep your monthly bill under \$100 per environment without sacrificing reliability.

Quick start

You'll need a few tools installed before you begin:

1. **Get your tools:** Install OpenTofu 1.9.0+, the AWS CLI, and the GitLab CLI on your machine.
2. **Handle your credentials:** Set up your AWS credentials using the AWS CLI or environment variables so the tools can talk to your account.

3. Plan your changes: Run `tofu plan` locally. It's the best way to double-check your work before you commit anything.

3.2 Architecture

3.2.1 How it's built

We've organized the infrastructure into three clear layers. Each one is tuned for performance, security, and keeping costs low.

3.2.2 Network layer

This layer handles all traffic routing and connectivity. At the foundation, you'll work with an Amazon VPC that acts as your private playground in the cloud, keeping your resources isolated and safe across multiple availability zones in production or a single AZ in development. The Internet gateway serves as the front door that lets traffic in from the internet to your public subnets, while the NAT gateway provides a private exit that lets your backend services reach the internet without letting the internet reach them directly.

Traffic distribution happens through an Application Load Balancer (ALB) that routes requests to your ECS services and handles SSL termination automatically, so you don't have to worry about certificate management in your application code. Sitting in front of everything is Amazon CloudFront, a global network that delivers your content fast by handling your frontend files and proxying your API requests, eliminating CORS issues by serving everything from one domain.

To minimize costs, you'll use VPC endpoints that act like secret tunnels to other AWS services (S3, ECR, CloudWatch, Secrets Manager). They're much cheaper than routing everything through the NAT gateway and can reduce your NAT costs by 50-90%.

3.2.3 Application layer

This is where your code runs. Amazon ECS Fargate serves as your serverless container orchestration platform, running your containers without you having to manage any servers. Within ECS, you'll have an API service that stays busy handling all your REST API requests, and a worker service that takes care of heavy lifting in the background.

Amazon ECR provides a private home for your Docker images, automatically scanning them for vulnerabilities and supporting immutable tags to ensure deployment integrity. For event-driven workloads, AWS Lambda functions handle cost monitoring and automation tasks like calculating Bedrock usage costs. Your observability needs are covered by Amazon CloudWatch, which acts as your eyes and ears by collecting logs, tracking metrics, and sending you alerts if something goes wrong.

3.2.4 Data layer

Your data lives here, encrypted and secure. Amazon RDS for PostgreSQL serves as your managed database that handles your data, supporting Multi-AZ high-availability for production and single AZ for development to keep costs down. Amazon S3 provides object storage for your static files, logs, and backups, with lifecycle policies that automatically move old logs to cheaper storage tiers.

Security is handled through AWS Secrets Manager, which acts as a secure vault for your database passwords and API keys that your ECS tasks access at runtime using IAM roles. AWS KMS manages the customer-managed keys used to encrypt your RDS database at rest, giving you full control over your encryption strategy.

3.3 Component relationships and data flow

Here's how everything connects together. This diagram shows the complete architecture from GitLab CI/CD through to production deployment:

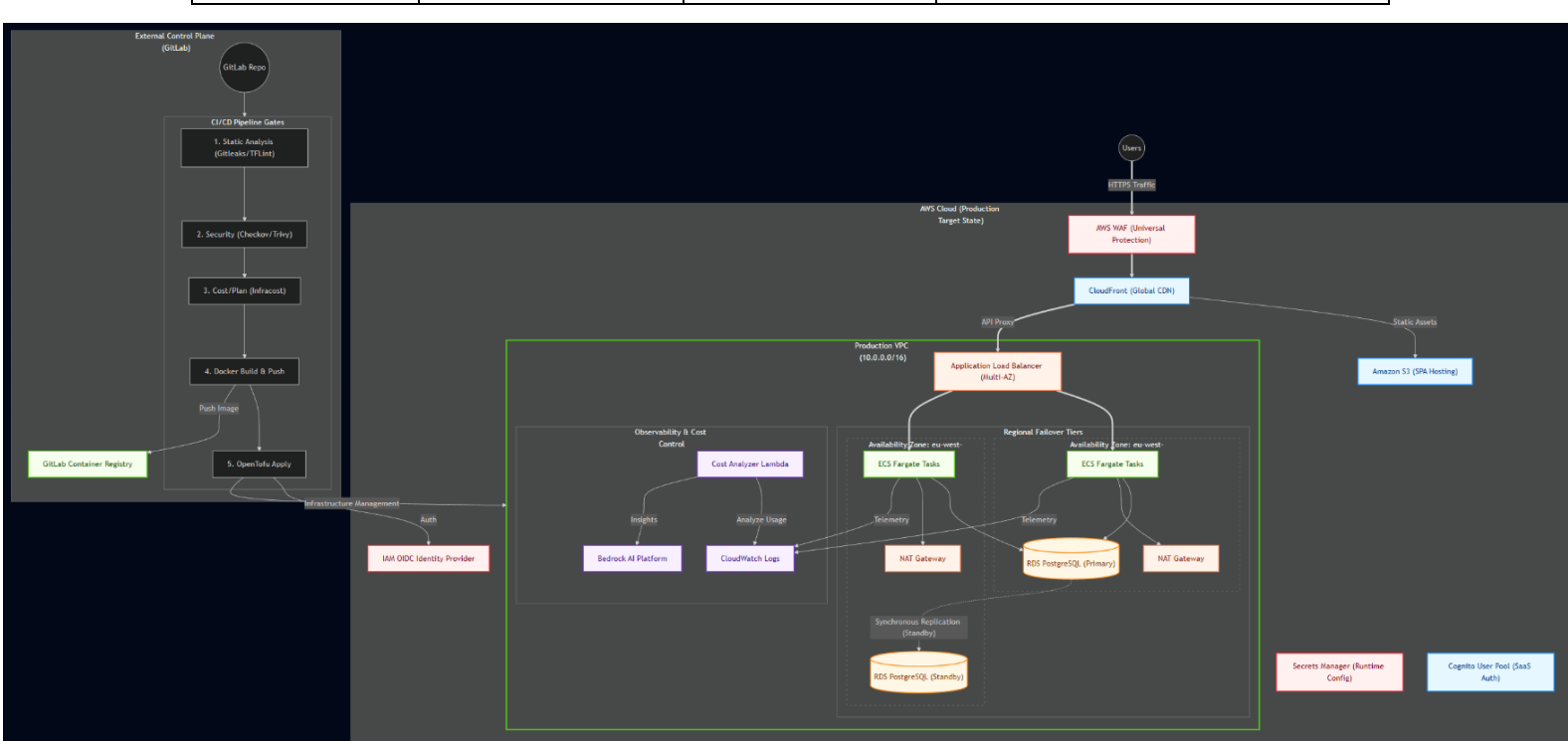
3.3.1 Key design decisions:

- **Single-entry-point architecture:** CloudFront eliminates CORS issues by serving both frontend and backend from the same domain.
- **Private-by-default:** All compute resources run in private subnets with controlled egress.
- **Cost-optimized networking:** VPC endpoints reduce NAT Gateway data transfer costs significantly.
- **Immutable infrastructure:** Container images are immutable, and deployments use blue-green patterns via ECS for zero-downtime updates.

3.3.2 Architectural decisions

We've made some specific choices for this project to prioritize security and cost-efficiency. Here's a quick look at why we're doing things "The Icarus Way" instead of just following the crowd.

Feature	Standard approach	The Icarus way	Why it matters
Authentication	Static IAM keys	OIDC/Keyless auth	You don't have to worry about long-lived secrets leaking from your CI/CD.
State management	S3 + DynamoDB	GitLab HTTP backend	It's easier to set up and ties your infrastructure state directly to your code.
Network cost	NAT gateway for everything	VPC endpoints	Keeps your AWS traffic inside the network, which cuts your NAT bill by 90%.
CORS strategy	Multi-domain setup	CloudFront proxy	One domain for everything. No CORS errors, no fuss.
Deployment	Simple ECR push	Direct ECS update	The pipeline does an atomic update, so your users never see downtime.
Monitoring	Basic alerts	Granular FinOps	You'll get alerts for specific costs like AI token usage or data transfers.



3.4 Backend application

The backend is a Node.js and TypeScript app that connects your logic unit with a readability layer.

3.4.1 What's inside

The backend architecture centers around a logic unit that acts as the "brain" following rules to process insurance claims. This connects to a readability layer that uses AWS Bedrock to turn the logic unit's output into something humans can actually read and use. For data access, we use Prisma as the ORM to talk to our PostgreSQL database, keeping everything structured and easy to manage.

3.4.2 Cool things we've done

We've implemented several features that make your life easier. The backend uses task-specific permissions through IAM roles for ECS tasks, which means the app gets only the permissions it needs at runtime (like accessing Bedrock or Secrets Manager), so you don't have to manage any secret access keys.

Plus, it is build using arm architecture to save cost on the infrastructure about - 20%

For database management, we run smart migrations as temporary ECS tasks that use the same network settings as your API, so you'll never have connection issues during deployments. When things go wrong, our auto-debugging feature kicks in and the pipeline automatically grabs the last 100 lines of logs from CloudWatch and shows them to you in the pipeline output.

3.5 Frontend application

Our frontend is a fast, secure React app that focuses on a great user experience.

What's inside

The frontend leverages React & Vite for their speed and excellent developer experience, with Vite providing lightning-fast hot module replacement during development. For hosting, we keep things simple by storing static files on Amazon S3 and delivering them globally through CloudFront with edge caching. User security is handled through Amazon Cognito, which manages authentication and authorization safely.

3.5.1 How it stays flexible

Several features make development and deployment easier. The deployment pipeline uses auto-discovery to find Cognito IDs and API endpoints based on your environment's unique suffix, so you don't need to hardcode anything. We've implemented a no-CORS design by setting the API URL to be relative—since CloudFront proxies those requests to the backend ALB, everything stays on one domain and CORS issues simply disappear. Security is baked into the build process through hardened builds using Chainguard images during Docker builds, minimizing the attack surface of your build environment.

3.6 State management strategy

Backend configuration deep-dive

We manage our infrastructure state using GitLab's native HTTP backend. It keeps everything in sync without you having to worry about it.

```
terraform {  
  backend "http" {}  
}
```

3.6.1 Why we like the HTTP backend

The HTTP backend offers several advantages that make infrastructure management smoother. Your state is stored securely in GitLab rather than in local `.tfstate` files, so you don't have to worry about losing them or accidentally committing them to version control. Concurrency is handled perfectly with automatic locking, meaning you and your teammates can work on the infrastructure at the same time without breaking things.

Since the backend is tied to GitLab, you get a full version history of your state changes right alongside your code commits, making it easy to understand what changed and when. Security is built in—everything is encrypted, and we control access using GitLab's own permission system. There's no need to manage separate S3 bucket policies or DynamoDB tables.

3.6.2 Keeping things locked and safe

The backend prevents conflicts through several mechanisms. It automatically locks the state during a tofu apply using HTTP headers to manage locking transparently, so two people can't change the same thing at once. We use separate state names for development and production, which means you'll never accidentally break production while testing something in dev. If a pipeline fails mid-execution, the setup is smart enough to release the state lock after a timeout, so you aren't stuck waiting.

3.6.3 Our workspace strategy

We use workspaces to keep our environments organized:

- **Development (`test`)**: An ephemeral environment for testing your latest changes. You can manually destroy it after validation to save costs.
- **Production (`production`)**: The real deal, serving live users. This is a persistent environment.

3.6.4 Why we use multiple environments

This multi-environment approach gives you several benefits. Safety comes first—you'll always have a chance to catch bugs in development before they reach your users, since every change goes through the dev environment first. For cost management, you can spin up dev environments when you need them and tear them down when you're done, though test infrastructure remains active until manually destroyed. The isolated workspace approach means several people can test their own merge requests at the same time without stepping on each other's toes, since each MR is completely isolated.

3.7 Security by design

We take security seriously. Here's how we keep the Icarus infrastructure safe from every angle.

3.7.1 Encrypting your data

Every data store uses encryption at rest to protect your information. Amazon S3 buckets use SSE-S3 encryption with AES256—it's safe, simple, and cheaper than using KMS for every object while still providing strong encryption. Your Amazon RDS database takes encryption more seriously with a customer-managed KMS key, giving you full control over encryption policies and key rotation schedules.

Even your operational CloudWatch Logs are encrypted at rest using server-side encryption to ensure they cannot be accessed in plaintext. Amazon ECR automatically encrypts every container image you store with AES256, protecting your application artifacts from unauthorized access.

3.7.2 Permissions that make sense

We follow the principle of least privilege everywhere. Our ECS tasks and Lambda functions only get the exact permissions they need—for example, ECS task roles can access Bedrock and Secrets Manager but nothing else. This minimizes the potential damage if a service is compromised.

For CI/CD authentication, we use OpenID Connect (OIDC) instead of long-lived access keys, generating short-lived, project-specific tokens that expire quickly. You won't find any hardcoded AWS keys in our code, repository, or pipeline settings. All authentication goes through IAM roles or temporary tokens, keeping your credentials secure.

3.7.3 Scanners that watch your back

Every time you commit code, several security tools jump into action automatically. GitLeaks checks your code to make sure you didn't accidentally include any secrets or API keys in your commits. TFLint validates that your infrastructure code follows OpenTofu and Terraform best practices and validates syntax. Checkov scans for over 190 different security misconfigurations in your Infrastructure as Code before deployment. Trivy looks for vulnerabilities in your container images and OS packages, detecting known security risks early in the development lifecycle.

If any of these scanners find an issue, the pipeline fails immediately, preventing insecure code from being deployed.

3.8 Network security

Multiple layers protect your infrastructure from threats. Your backend and database live in private subnets, making them unreachable from the public internet—only the ALB and CloudFront can access them. AWS WAF acts as a shield for your CloudFront distribution, blocking common web attacks and malicious traffic before it reaches your application layer. We've set up tight security groups with strict rules that only allow the specific traffic we expect, with minimal inbound access and tightly controlled outbound access.

3.9 Operational guide

3.9.1 Local setup

You'll need these tools installed before you can work with the infrastructure locally.

3.9.2 What you'll need

Make sure you have these installed:

- OpenTofu 1.9.0+ (or Terraform 1.5+)
- The AWS CLI (properly configured)
- The GitLab CLI (glab)
- Docker (if you want to test containers locally)

3.9.3 Your environment variables

Create a .env file in the project root with these variables to get started:

```
AWS_ACCESS_KEY_ID=your_access_key
AWS_SECRET_ACCESS_KEY=your_secret_key
# Only if you're using temporary session keys:
AWS_SESSION_TOKEN=your_session_token
AWS_DEFAULT_REGION=eu-north-1
```

3.9.4 How to initialize

Just follow these steps to get your project ready:

1. **Connect to GitLab:** `glab opentofu init -R 'it-factory-thomas-more/project-4-0/25-26/m2-icarus/icarus-infrastructure' 'default'`
2. **Load your variables:** `set -a; source .env; set +a`

3. **Go to the terraform folder:**`cd terraform`
4. **Set up your local vars:**`cp terraform.tfvars.example local.tfvars`Don't forget to edit `local.tfvars` with your own settings!
5. **Run a plan:**`tofu plan -var-file="local.tfvars"`

3.9.5 Your daily workflow

Here's the typical development cycle you'll follow:

1. Tweak your infrastructure code in the `terraform/` folder.
2. Run `tofu plan -var-file="local.tfvars"` locally to make sure it's doing what you expect.
3. Commit your changes and push them to a new merge request.
4. Our CI/CD will spin up a preview environment automatically.
5. Check out your changes in the MR environment using the CloudFront URL posted in the merge request.
6. Once everything looks good, merge to main to deploy to production.

3.10 CI/CD pipeline

We've built a three-gate system to make sure only high-quality, secure code makes it to production.

3.10.1 Gate 1: Verification (automatic)

Every time you push code, we automatically check for:

- **Secret scanning:** GitLeaks prevents credential leaks by scanning your commits for exposed secrets.
- **Code quality:** TFLint and Checkov validate your IaC against best practices and security requirements.
- **Security vulnerabilities:** Trivy scans for known vulnerabilities in container images and infrastructure definitions.
- **Cost estimation:** Infracost provides budget impact analysis, showing you exactly how much your changes will cost before deployment.
- **OWASP-zap:** DAST tool for security testing and verification.

If any of these checks fail, the pipeline stops immediately and shows you what needs to be fixed.

3.10.2 Gate 2: Production promotion (post-merge)

After you merge to main, you have manual control over production:

- **Manual approval:** Production deployment requires explicit approval to prevent accidental changes.
- **State separation:** Production uses its own dedicated state, completely independent of development.
- **Rollback capability:** Previous state versions enable quick reversion if something goes wrong.
- **Monitoring integration:** Deployment triggers enhanced monitoring and alerting.
- **Manual destroy:** Both dev and prod pipelines include a final destroy step that requires manual trigger to clean up resources.

3.11 Safety guardrails

The infrastructure is protected by several built-in safety features. Development and production environments never share state, preventing cross-environment interference. Production deployments require manual approval to avoid accidental changes. Cost monitoring with automated budget alerts triggers at the 80% threshold to prevent unexpected charges. Security gates cause the pipeline to fail immediately if any security violation is detected during the verification phase.

Cost optimization

Cost efficiency is a central design principle of the Icarus infrastructure. The architecture balances performance, high availability, and security while maintaining a strict monthly budget target of approximately \$100 per environment.

Cost breakdown

The development environment prioritizes cost savings with single-AZ deployments and Spot instances, while the production environment prioritizes high availability and resilience. Here's the expected monthly cost distribution:

Component	Dev environment	Prod environment	Monthly cost
ECS Fargate	Spot instances	Standard (HA)	\$15-25
RDS	Single AZ	Multi-AZ	\$20-40
NAT Gateway	Single AZ	Single AZ	\$32 + data
CloudFront	Global	Global	\$5-10
Monitoring	Basic	Enhanced	\$5-10
Total			\$77-117

Budget controls

We've implemented several mechanisms to keep costs under control. S3 lifecycle policies automatically move old logs to cheaper storage tiers (like Glacier) after 30 days to minimize storage costs. Lambda concurrency limits cap function executions to prevent runaway costs from unexpected traffic spikes.

NAT Gateway monitoring

tracks data transfer patterns using dedicated CloudWatch metrics to detect cost anomalies before they become expensive problems. Budget alerts through AWS Budgets send email notifications when you hit 80% utilization, giving you time to investigate before hitting the limit.

Cost optimization strategies

You'll save money through several architectural choices we've made. By routing AWS service traffic through VPC endpoints instead of NAT gateways, you'll cut NAT data transfer costs by 50-90%. Development environments use ECS Fargate Spot capacity, which can save up to 70% compared to on-demand pricing, though production stays on standard capacity for reliability.

Development of RDS instances run in a single availability zone, cutting database costs in half while still providing adequate reliability for testing. In production, we use Multi-AZ for high availability. Finally, aggressive CloudFront caching policies reduce backend requests and minimize data transfer costs from your origin.

Monitoring and observability

Observability is fully integrated into the infrastructure to provide real-time visibility into performance, security, and financial metrics.

Unified monitoring dashboard

Amazon CloudWatch acts as the central observability platform, providing comprehensive monitoring across all services. You'll see API performance metrics including latency percentiles, request rates, and error rates for your REST API. ECS service health monitoring covers task count, CPU/memory utilization, and scaling metrics, while database performance indicators track connection counts, query latency, and replication lag for Multi-AZ deployments.

Cost tracking gives you real-time metrics for Bedrock usage and NAT Gateway data transfer, helping you spot expense trends early. Security events monitoring captures AWS WAF blocks, authentication failures, and anomalous traffic patterns. This unified dashboard enables rapid diagnosis of operational issues and supports informed decision-making.

Cost monitoring and analysis

Beyond standard monitoring, the infrastructure includes automated cost intelligence mechanisms. A dedicated Lambda function calculates per-request usage costs for Amazon Bedrock services, providing granular insight into AI-related spending. Athena queries analyze NAT Gateway traffic patterns to identify inefficient routing or excessive outbound traffic that's driving up costs. AWS Budgets integration provides automated email alerts to ensure financial thresholds are continuously tracked.

When we'll alert you

You'll receive notifications when these thresholds are exceeded:

- **API errors:** 5XX error rate exceeds 1% for more than 5 minutes.
- **High AI spend:** Bedrock daily spend goes over \$50.
- **Network spikes:** NAT Gateway data transfer shows unexpected spikes.
- **Budget threshold:** You've reached 80% of your monthly budget allocation

3.12 Logs and debugging

When you need to troubleshoot issues, here's where to find the information you need.

ECS logs

ECS application logs can be viewed in CloudWatch Logs under groups such as:
`/ecs/icarus-api`

You can also stream logs in real-time using the AWS CLI:

```
$ aws logs tail /ecs/icarus-api --follow
```

Lambda logs

Lambda execution logs are available in:
`/aws/lambda/icarus-cost-calculator`

Review these logs for cost calculation or execution errors. You can access them through the CloudWatch console or CLI.

VPC flow logs

Network traffic data can be analyzed via VPC Flow Logs stored in the designated S3 bucket. This is useful for diagnosing NAT Gateway-related networking issues or investigating security incidents.

Terraform state

All infrastructure state changes are managed through GitLab. Review CI/CD pipeline artifacts and logs for detailed information about plan or apply operations. You can also inspect state directly using:

```
$ tofu state list  
$ tofu state show <resource_name>
```

3.13 Troubleshooting

Common issues you might run into

Here are solutions to the most frequent problems:

"State is locked"

This usually means a pipeline is still running or a previous run didn't clean up properly. Wait a few minutes for the lock to expire automatically, or use `tofu force-unlock` if you're absolutely sure nothing else is using it:

```
$ tofu force-unlock <lock_id>
```

High NAT costs

Double-check that your big AWS services (like S3, ECR, CloudWatch) are using VPC endpoints. If they aren't, they'll route through the NAT gateway and your bill will climb quickly. You can verify this by checking VPC Flow Logs for traffic to AWS service IP ranges.

ECS won't deploy

Check your ECR images and look for `CannotPullContainerError` in the ECS task logs. It's usually a permission issue (ECS task execution role missing ECR access) or a network issue (no route to ECR or VPC endpoint misconfigured).

CloudFront errors

Check your S3 bucket policies and make sure your SSL certificate is still valid. Also verify that your CloudFront origin settings match your ALB configuration and that security groups allow traffic from CloudFront.

3.14 Glossary

If some of these terms are new to you, here's a quick breakdown:

- **IaC (Infrastructure as Code):** This just means you're managing your servers and network using code files instead of clicking around in a dashboard.
- **OpenTofu:** A great tool for building and changing infrastructure safely. It's a fork of Terraform and is fully compatible with Terraform syntax.
- **VPC (Virtual Private Cloud):** Your own private section of the AWS cloud, isolated from other customers.
- **ECS Fargate:** A way to run containers without having to manage any of the underlying servers. AWS handles all the infrastructure for you.
- **NAT Gateway:** A bridge that lets your private resources talk to the internet securely without exposing them to inbound internet traffic.
- **OIDC (OpenID Connect):** A secure way for GitLab to talk to AWS using temporary permissions instead of permanent passwords or access keys.
- **WAF (Web Application Firewall):** A digital shield that protects your site from common attacks like SQL injection and cross-site scripting.
- **CloudFront:** AWS's global content delivery network. It makes your site fast no matter where your users are by caching content at edge locations worldwide.
- **Multi-AZ:** Deploying resources across multiple availability zones for high availability. If one zone fails, your service keeps running.
- **VPC Endpoints:** Private connections between your VPC and AWS services that don't require internet gateway or NAT gateway access.

Conclusion

The Cloudway project demonstrates how Artificial Intelligence, application services, and cybersecurity measures can be integrated into a single, ICARUS platform.

Each component AI processing, backend application logic, and security controls works together to provide a secure, reliable, and efficient environment for users to make requests.